PRACTICAL SESSION 9

Task 1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
(base) saiabburi@Charishma-MacBook-Air Practical9 % echo "Operating Systems Session 9 file copy practical." > input.txt
(base) saiabburi@Charishma-MacBook-Air Practical9 % clang -Wall -Wextra file_copy.c -o file_copy
(base) saiabburi@Charishma-MacBook-Air Practical9 % ./file_copy input.txt
Low-level system-call copy: successful
Standard-library copy: successful
Low-level CPU time: 0.000281 seconds
Library CPU time: 0.000189 seconds
Compare copied files using cmp.
(base) saiabburi@Charishma-MacBook-Air Practical9 % cmp input.txt copy_low_level.txt
(base) saiabburi@Charishma-MacBook-Air Practical9 % cmp input.txt copy_library.txt
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat copy_low_level.txt
Operating Systems Session 9 file copy practical.
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat copy_library.txt
Operating Systems Session 9 file copy practical.
(base) saiabburi@Charishma-MacBook-Air Practical9 %
```

```
  UW PICO 5.09                                                File: file_copy.c


#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
#include <errno.h>

#define BUFFER_SIZE 4096

int copy_low_level(const char *source, const char *destination) {
    int input = open(source, O_RDONLY);
    if (input == -1) {
        perror("open source");
        return -1;
    }

    int output = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output == -1) {
        perror("open destination");
        close(input);
        return -1;
    }

    /* Demonstrate lseek() */
    if (lseek(input, 0, SEEK_SET) == (off_t)-1) {
        perror("lseek");
        close(input);
        close(output);
        return -1;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    int failed = 0;

    while ((bytes_read = read(input, buffer, sizeof(buffer))) > 0) {
        ssize_t total_written = 0;

        while (total_written < bytes_read) {
            ssize_t bytes_written = write(
                output,
                buffer + total_written,
                (size_t)(bytes_read - total_written)
            );

            if (bytes_written <= 0) {
                perror("write");
                failed = 1;
                break;
            }

            total_written += bytes_written;
        }

        if (failed) {
            break;
        }
    }

    if (bytes_read < 0) {
```

```
  UW PICO 5.09                                                    File: file_copy.c

        perror("read");
        failed = 1;
    }

    if (close(input) == -1) {
        perror("close input");
        failed = 1;
    }

    if (close(output) == -1) {
        perror("close output");
        failed = 1;
    }

    return failed ? -1 : 0;
}

int copy_standard_library(const char *source, const char *destination) {
    FILE *input = fopen(source, "rb");
    if (input == NULL) {
        perror("fopen source");
        return -1;
    }

    FILE *output = fopen(destination, "wb");
    if (output == NULL) {
        perror("fopen destination");
        fclose(input);
        return -1;
    }

    char buffer[BUFFER_SIZE];
    size_t bytes_read;
    int failed = 0;

    while ((bytes_read = fread(buffer, 1, sizeof(buffer), input)) > 0) {
        if (fwrite(buffer, 1, bytes_read, output) != bytes_read) {
            perror("fwrite");
            failed = 1;
            break;
        }
    }

    if (ferror(input)) {
        perror("fread");
        failed = 1;
    }

    if (fclose(input) == EOF) {
        perror("fclose input");
        failed = 1;
    }

    if (fclose(output) == EOF) {
        perror("fclose output");
        failed = 1;
    }

    return failed ? -1 : 0;
}

int main(int argc, char *argv[]) {
```

```
  UW PICO 5.09                                                File: file_copy.c

    if (argc != 2) {
        printf("Usage: %s source_file\n", argv[0]);
        return 1;
    }

    const char *source = argv[1];
    const char *low_destination = "copy_low_level.txt";
    const char *lib_destination = "copy_library.txt";

    clock_t start = clock();

    if (copy_low_level(source, low_destination) != 0) {
        return 1;
    }

    clock_t middle = clock();

    if (copy_standard_library(source, lib_destination) != 0) {
        return 1;
    }

    clock_t end = clock();

    printf("Low-level system-call copy: successful\n");
    printf("Standard-library copy: successful\n");
    printf("Low-level CPU time: %.6f seconds\n",
           (double)(middle - start) / CLOCKS_PER_SEC);
    printf("Library CPU time: %.6f seconds\n",
           (double)(end - middle) / CLOCKS_PER_SEC);

    printf("Compare copied files using cmp.\n");
    return 0;
}
```

Task 2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
  UW PICO 5.09                        File: io_redirect.c


#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int input_fd;
    int output_fd;
    char buffer[1024];
    ssize_t bytes_read;

    input_fd = open("input.txt", O_RDONLY);
    if (input_fd == -1) {
        perror("open input.txt");
        return 1;
    }

    output_fd = open("redirected_output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd == -1) {
        perror("open redirected_output.txt");
        close(input_fd);
        return 1;
    }

    /* Redirect standard input to input.txt */
    if (dup2(input_fd, STDIN_FILENO) == -1) {
        perror("dup2 stdin");
        close(input_fd);
        close(output_fd);
        return 1;
    }
```

```
 UW PICO 5.09                          File: io_redirect.c

        return 1;
    }

    /* Redirect standard output to redirected_output.txt */
    if (dup2(output_fd, STDOUT_FILENO) == -1) {
        perror("dup2 stdout");
        close(input_fd);
        close(output_fd);
        return 1;
    }

    close(input_fd);
    close(output_fd);

    while ((bytes_read = read(STDIN_FILENO,
                              buffer, sizeof(buffer))) > 0) {
        ssize_t offset = 0;

        while (offset < bytes_read) {
            ssize_t written = write(STDOUT_FILENO,
                                    buffer + offset,
                                    (size_t)(bytes_read - offset));

            if (written <= 0) {
                perror("write");
                return 1;
            }

            offset += written;
        }
    }

    if (bytes_read < 0) {
        perror("read");
        return 1;
    }

    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air Practical9 % nano io_redirect.c
(base) saiabburi@Charishma-MacBook-Air Practical9 % clang -Wall -Wextra io_redirect.c -o io_redirect
(base) saiabburi@Charishma-MacBook-Air Practical9 % ./io_redirect
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat redirected_output.txt
Operating Systems Session 9 file copy practical.
(base) saiabburi@Charishma-MacBook-Air Practical9 % echo "Hello from Terminal" > shell_output.txt
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat shell_output.txt
Hello from Terminal
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat < input.txt
Operating Systems Session 9 file copy practical.
(base) saiabburi@Charishma-MacBook-Air Practical9 % echo "Another line" >> shell_output.txt
(base) saiabburi@Charishma-MacBook-Air Practical9 % cat shell_output.txt
Hello from Terminal
Another line
(base) saiabburi@Charishma-MacBook-Air Practical9 %
```